



Docker Step 5 — Custom Networks

Isolating the tiers of a multi-service app, so each service only reaches what it should

Project: `step5-custom-networks` / `step5-practice` • Next.js → Express → Redis • Generated 29 June 2026

What this guide is. A beginner-friendly walkthrough of *custom* Docker networks — going beyond the single auto-created network to give each tier of your app its own network. The goal: stop the frontend from reaching the database directly. Every idea is grounded in the real output you saw on your own machine, and we prove the isolation works at the end.

Prerequisite: the companion guide "*Docker's Default Network*", which covers networks, drivers, the bridge, DNS/service discovery, IPs, subnets, and the `/16` notation. This guide builds on those.



Contents

1. Why custom networks? (the problem)
2. The goal: isolate the tiers
3. The two-network design
4. The Compose syntax
5. Who joins which network (and why)
6. How the frontend still reaches the backend
7. The big reveal: one container, two IPs
8. Two separate subnets
9. Proving the isolation (hands-on)
10. Default vs custom — side by side
11. Command gotchas you hit
12. Bonus: `internal: true`
13. Commands cheat-sheet
14. Key takeaways & glossary

1. Why custom networks? (the problem)

By default, Compose puts **every service on one shared network** (`<project>_default`). That's convenient — but it means *any* container can reach *any* other. In particular, your **frontend can talk directly to Redis**, even though it never should.

You proved this on the default network: from inside the frontend, a Redis **PING** came straight back:

```
# on the DEFAULT network (everything together)
$ docker compose exec frontend getent hosts redis
172.18.0.2    redis          # ← frontend can SEE redis
# ...and a TCP PING returned: +PONG ← frontend can TALK to redis
```

🚩 **The risk:** the database should only be reachable by the backend. On a flat network there's no wall — if the frontend (the most exposed tier) were ever compromised, it has a direct line to your data.

2. The goal: isolate the tiers

A three-tier app has a natural "chain of trust":

```
browser → frontend → backend → redis
(public)  (UI)      (logic)  (data)
```

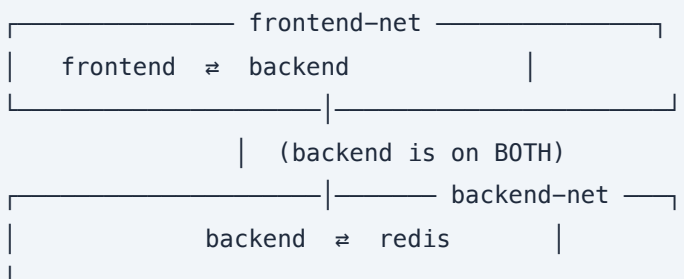
Each tier should only talk to its **immediate neighbour**:

- frontend → backend ✅ (needed)
- backend → redis ✅ (needed)
- frontend → redis ❌ (should be blocked)

Custom networks let us enforce exactly that — a principle called **defense in depth**: even inside your app, services can't reach things they don't need.

3. The two-network design

We create **two** networks and make the backend the bridge between them:



Service	Network(s)	Can reach
frontend	frontend-net	backend ✓ · redis ✗
backend	frontend-net + backend-net	frontend ✓ · redis ✓
redis	backend-net	backend ✓ (only)

4. The Compose syntax

Two parts: (a) a `networks:` key on each service saying which networks it joins, and (b) a top-level `networks:` block that declares the networks exist.

```
services:
  frontend:
    build: ./frontend
    ports: ["3000:3000"]
    networks:
      - frontend-net          # frontend joins ONE network

  backend:
    build: ./backend
    networks:
      - frontend-net          # backend joins BOTH
      - backend-net

  redis:
    image: redis:7
    networks:
      - backend-net           # redis joins ONE network

# Declare the networks (Docker creates them on `up`)
networks:
  frontend-net:
  backend-net:
```

Two halves to remember: the `networks:` under each service = "which networks this service plugs into." The `networks:` at the bottom = "these networks exist." You need both.

5. Who joins which network (and why)

The rule that decides everything:

 **The rule:** a container can only reach others on a network it **shares**. So each service joins exactly the networks it needs — no more.

- **frontend** only needs the backend → 1 network (`frontend-net`).
- **backend** needs the frontend AND redis → 2 networks (both).
- **redis** only needs the backend → 1 network (`backend-net`).

The backend is the only service that needs to reach *both* tiers, so it's the only one on both networks — the **bridge**.

6. How the frontend still reaches the backend

A natural question: the frontend isn't on `backend-net` — so how does it talk to the backend at all?

Answer: through the network they SHARE — `frontend-net` . Both the frontend and the backend are members of `frontend-net` , and that one shared network is all they need.

💡 **Key rule:** two containers can communicate over *any* network they **both** belong to. They only need **one** network in common. The backend's extra membership in `backend-net` is a separate, private line to Redis — invisible to the frontend.

frontend —(frontend-net, shared)→ backend —(backend-net, private)→ redis

7. The big reveal: one container, two IPs

When you inspected your two networks on `step5-practice`, the proof was right there. The **backend** appeared in **BOTH** networks — with a **DIFFERENT** IP in each:

Network	Subnet	Members & their IPs
<code>backend-net</code>	172.18.0.0/16	redis → 172.18.0.2 backend → 172.18.0.3
<code>frontend-net</code>	172.19.0.0/16	backend → 172.19.0.2 frontend → 172.19.0.3

Look at the backend: 172.18.0.3 on backend-net AND 172.19.0.2 on frontend-net. **A container on two networks gets a separate address on each.** That's literally what makes it the bridge — one foot (and one IP) in each network.

Meanwhile **frontend** only has 172.19.x.x and **redis** only has 172.18.x.x — they have no address on each other's network at all.

8. Two separate subnets

Notice the two networks have **different subnets**:

```
frontend-net : 172.19.0.0/16 (addresses 172.19.x.x)
backend-net  : 172.18.0.0/16 (addresses 172.18.x.x)
```

Docker gives **each network its own separate address space**. These aren't sub-ranges of one another — they're two independent "neighbourhoods." A device in one has no address in the other.

 **Analogy:** `frontend-net` and `backend-net` are two different neighbourhoods. frontend lives only in one, redis only in the other. The backend owns a house in *both* (a different street address in each), so it can carry messages between them. Frontend and redis live in different neighbourhoods with no road connecting them.

9. Proving the isolation (hands-on)

From inside the frontend container, check whether each name resolves. `getent hosts <name>` returns an IP if it resolves, and **nothing** if it can't.

```
$ docker compose exec frontend sh

# backend IS reachable (shared frontend-net)
# getent hosts backend
172.19.0.2    backend      ✔ resolves

# redis is NOT reachable (no shared network)
# getent hosts redis
#
                                ✖ empty = isolated!
# echo $?
2                                ← exit code 2 = name not found
```

💡 **The empty output IS the success.** Seeing `backend` resolve but `redis` come back empty proves the isolation: the frontend can talk to the backend but is completely walled off from Redis.

⚠ **Watch the spelling.** It's `getent hosts` (plural — same word as the `/etc/hosts` file). `host` (singular) gives "Unknown database"; a misspelling like `gentent` gives "not found" — neither is the real isolation result.

10. Default vs custom — side by side

	Default network (step4)	Custom networks (step5)
Networks	one (<code><project>_default</code>)	two (<code>frontend-net</code> , <code>backend-net</code>)
frontend → backend	✅ works	✅ works
backend → redis	✅ works	✅ works
frontend → redis	⚠️ works (no isolation)	✅ blocked (<code>ENOTFOUND</code>)
<code>getent hosts redis</code> from frontend	<code>172.18.0.2</code>	(empty, exit 2)

Same app, same code — the **only** difference is the network setup in `docker-compose.yml`.

11. Command gotchas you hit

Command	Problem	Fix
<code>docker network inspect</code>	"requires at least 1 argument" — no network named	name it: <code>docker network inspect <name></code>
<code>docker inspect network <name></code>	tried to inspect a thing literally called "network"	<code>docker network inspect <name></code> (network is a subcommand)
<code>getent host redis</code>	"Unknown database: host" — wrong database name	<code>getent hosts redis</code> (plural)
<code>gentent hosts redis</code>	"not found" — typo in the command	<code>getent hosts redis</code>

Pattern: commands that act on a specific thing (`inspect` , `exec` , `logs`) need you to *name* that thing. "additional properties / requires N arguments / unknown" errors almost always mean a typo or a missing target.

12. Bonus: `internal: true`

By default a custom network can still reach the **internet** (via the gateway). For a database, you often don't want that. Marking a network `internal` seals it off from the outside world entirely:

```
networks:
  frontend-net:
  backend-net:
    internal: true          # backend-net can't reach the internet
```

Now Redis (on `backend-net`) can talk to the backend but has **no route to the internet** — an extra layer of lockdown. The backend still reaches the outside via `frontend-net`. Optional, but a nice real-world touch.

13. Commands cheat-sheet

```
# List networks (find names/IDs)
docker network ls

# Inspect a network (members, subnet, gateway) – NAME IS REQUIRED
docker network inspect step5-practice_backend-net
docker network inspect 56e10ca72c7a      # short ID works too

# Just the members + IPs
docker network inspect step5-practice_backend-net \
  --format '{{range .Containers}}{{.Name}} {{.IPv4Address}}{{println}}{{end}}'

# Prove isolation from inside a container
docker compose exec frontend getent hosts backend  # should resolve
docker compose exec frontend getent hosts redis    # should be empty (exit 2)

# Clean up unused networks
docker network prune
```

14. Key takeaways & glossary

- The default network puts everything together → **no isolation**.
- Custom networks let each tier reach only its neighbour (**defense in depth**).
- Define them with a `networks:` key per service + a top-level `networks:` block.
- A container only needs **one shared network** with another to reach it.
- A container on multiple networks gets a **separate IP on each** — that's how the backend bridges two subnets.

- frontend and redis share no network → `getent hosts redis` returns nothing = isolated.

Term	Meaning
Custom network	a network you define yourself (vs the auto-created default)
Tier isolation	limiting each service to only the services it must reach
Bridge service	a container on two networks that links them (your backend)
Defense in depth	multiple layers of restriction, even inside your own app
Subnet	the address range a network owns (each network gets its own)
<code>internal: true</code>	seals a network off from the internet
<code>getent hosts</code>	look up a name→IP; empty result = doesn't resolve

Docker Step 5 — Custom Networks • Next.js → Express → Redis • Isolating tiers with two networks and a bridge
The frontend can't reach Redis anymore — and that's exactly the point. 🗝️